CANCUN
Software Engineering to Secure Large-Scale Open-Source Software

Securing the Open-Source Software Supply Chain,
Hunting Security and Privacy Bugs at Ecosystem Scale.

Inria domain: *Distributed programming and Software engineering*
IRISA department 4: *Language and software engineering*

Inria, University of Rennes, CNRS, IRISA
https://bagad-team.github.io/

Abstract

Modern software is no longer simply written and compiled; it is assembled. A single application recursively integrates thousands of components that its developers did not write, did not build, and frequently cannot enumerate. Every property that classical software engineering took for granted—that the source you read is the code that runs, that the build is a function of the source tree, that the set of components is knowable by inspection—has become an open question that requires evidence rather than assumption. This is the root cause of the software supply chain crisis, and it is a software engineering problem that leads into a security one.

CANCUN addresses this problem along two coupled axes. **Axis 1, Software Supply Chain Security**, establishes what a software system is actually made of: it builds evidence-based, formally grounded models of composition that span all forms of code reuse rather than declared dependencies alone, and it uses those models to specify distribution mechanisms that are secure by construction. **Axis 2, Hunting Security and Privacy Bugs at Scale**, finds what is wrong with that composition (known vulnerabilities propagating through undeclared reuse, privacy defects in APIs and device fingerprinting vectors in platform interfaces), and the structural health signals that determine where finite auditing effort should be spent.

The two axes share a foundation—the global graph of public source code, registry and build observations materialised by Software Heritage. Axis 1 determines what is in a system and how it got there, and Axis 2 determines what is wrong with it and how much that matters. The team is anchored in preliminary results that already demonstrate this coupling at ecosystem scale, in strategic partnerships with Software Heritage and ANSSI, and in a direct line of work on supply chain security. Our objective is to give Europe the capability to audit, repair, and certify the open-source software on which its public infrastructure depends.

Contents

1 Team composition

The members of CANCUN are located at Inria/IRISA, Rennes, France.

1.1 Permanent members (alphabetical order)

Name	Position	Affiliation
Olivier Barais	Professor	Université de Rennes
Stéphanie Challita	Associate Professor	Université de Rennes
Walter Rudametkin (**head**)	Professor	Université de Rennes, Institut Universitaire de France
Gerson Sunye	Associate Professor	Université de Rennes
Olivier Zendra	Researcher	Inria

1.2 Non-permanent members (alphabetical order)

Name	Period	Position	Affiliation
Valere Billaud	2025-2028	PhD student	Université de Rennes
Killian Callac	2026-2029	PhD student	INRIA
Nicolo Cavalli	2024-2027	PhD student	INSA Rennes (with the EPI KHORA)
Malvin Chevallier	2025-2028	PhD student	INSA Rennes
Haitam El Hayani	2024-2027	PhD student	Univ Rennes
Josselin Enet	2026-2028	Postdoc	Univ Rennes
Mathieu Goessens	2025-2026	Software Engineer	INRIA
Lise Lahoche	2023-2026	PhD student	INRIA
Gabin Lavazais	2026-2029	PhD Student	Univ Rennes
Maïwenn Le Goasteller	2025-2028	PhD student	Univ Rennes (with the EPI KHORA)
Camille Molinier	2024-2027	PhD student	Univ Rennes (with the EPI Artishau)
Sylvain Plessis	2026-2029	PhD Student	Univ Rennes (with Orange)
Alan Prado	2025-2027	Software Engineer	INRIA
Sterenn Roux	2023-2026	PhD student	Univ Rennes (with the EPI PIRAT)
To be recruited	2026-2029	PhD Student	INRIA (with the EPI AVALON)

2 Context and objectives

2.1 Context

2.1.1 From compiling code to assembling artefacts

The unit of software production has changed, and most of our tools have not caught up. Software engineering was built around a model in which a developer writes source code and a compiler turns it into an executable. In that model, three properties hold by construction: the source you read is the code that runs, the build output is a function of the source code tree, and the set of components in the system can be determined by reading it. None of these properties hold today. A modern application is produced by recursively integrating artefacts that its developers did not write, did not build, and frequently cannot enumerate. These are known as dependencies, and a dependency may bring in further dependencies. Free and open-source software, deployed more often than not as dependencies, constitute 70–90 % of the code in a typical modern system [37]. 96 % of commercial codebases contain open-source components [30], and the economic contribution of open source to the European Union is estimated at €65–95 billion annually [40]. Cox frames the dynamic as *"For decades, software reuse was only a loftygoal. Now it's very real."* [16].

The difficulty is not that reuse is widespread. It is that reuse now happens through mechanisms with incompatible notions of provenance. Table 1 lists the classes of artefacts that a single application typically integrates. Each column entry differs, and no current representation covers more than two or three rows at once.

	Artefact class	**Provenance model**	**Enumerable from source?**
(a)	Declared dependencies	Registry plus version constraint solver	Yes, if a lock file exists
(b)	Transitive closure of dependencies	Resolver-dependent, differs across package managers	Only after resolution
(c)	Vendored or copy-pasted code	None: metadata stripped on copy	No
(d)	Forked upstream history	Commit graph, cross-forge, undeclared	Only with a global graph
(e)	Dynamically loaded modules	Resolved at run time, configuration-dependent	Undecidable in general
(f)	Prebuilt binaries, containers, wheels	Asserted by publisher, rarely reproducible	No
(g)	Build-time and install-time scripts	Arbitrary code execution, unmodelled	No
(h)	Machine-generated code (LLMs)	None: provenance destroyed at generation	No
(i)	Agentic contributions (autonomous PRs)	Attributed to an agent identity, not to training origin	Only if the agent identity is disclosed

Table 1: Classes of artefacts integrated by modern applications and their corresponding provenance model. The heterogeneity of the *Provenance model*, not scale alone, is why determining the composition of a system has become a measurement problem rather than a lookup.

This heterogeneity explains an otherwise puzzling observation: the standards intended to describe software composition have fragmented rather than converged, and the tools that implement them disagree. Determining what a piece of software is made of is no longer a matter of consulting a manifest. It is an empirical question. Answering it reliably at ecosystem scale is not the whole of this project, but it is its foundation: every security and privacy analysis in the research programme rests on knowing, with quantified confidence, what the system under analysis contains.

Stochastic reuse is no longer speculative. Two developments have turned the machine-generated rows of Table 1 from forward-looking entries into operational concerns. First, language models trained on open-source corpora now generate code at scale, through both interactive assistants and autonomous coding agents, reproducing patterns, and more importantly in our case, reproducing vulnerabilities learned from their training data without retaining any record of origin. Roughly 40 % of programs produced by a code assistant in security-relevant scenarios contain exploitable weaknesses [20], and developers assisted by such models write measurably less secure code while being more confident of its security [34]. About a fifth of the package names recommended by code-generating models do not exist, and a substantial fraction of these hallucinated names recur deterministically across runs [22], making them registrable in advance by adversaries—a vector now termed *slopsquatting*. Second, autonomous coding agents now generate and submit pull requests to open-source repositories with minimal human intervention, creating a new entry point into the supply chain in which code of unknown provenance is reviewed and merged under trust assumptions designed for human contributions. These developments reinforce our thesis. Determining what a system is made of has always been an empirical question; when part of the system is synthesised rather than written, the question becomes harder and the answer more consequential.

2.1.2 Evidence that the abstraction is broken

The argument above is not speculative. We summarise four measurements, three of them produced by members of this team, that show the foundations of current supply chain tooling to be unsound.

Tools cannot agree on what a project depends on. A *Software Bill of Materials* (SBOM) is a machine-readable inventory of the components in a software product; it is the artefact on which nearly all supply chain security practice, and soon European regulation, depends. We generated SBOMs for 2 050 JavaScript and 1 276 Rust projects using three widely deployed generators, and compared them against a reference we built ourselves by regenerating and parsing dependency lock files across nine format versions [4]. Coverage against that reference ranges from 32.7 % to 100 %, and the ranking of the tools *inverts* between the two ecosystems: the tool that performs best on Rust performs worst on JavaScript. Crucially, the divergences are explicable and systematic. They decompose into representation differences (the same component reported under an alias, with a peer-dependency suffix appended to its version, or prefixed by a directory path), deliberate design choices (whether development dependencies belong in the inventory at all), and outright implementation errors. When a project ships no lock file, which is common for libraries, two of the three tools produce a *silently empty* SBOM, with no warning. And none of them populate the *Supplier* field, although it is one of the minimum elements mandated by the United States NTIA [44] and reaffirmed in CISA's 2025 revision [17].

Package managers execute untrusted code by construction. When a developer runs `npm install`, the package manager may execute shell commands supplied by each package being installed. We measured the consequences across 3.56 million npm packages and 4 379 confirmed malicious ones [14]. A package that declares a `preinstall` script has a 29.9 % probability of being malicious and a relative risk of 427.9 compared with the ecosystem baseline. The same analysis shows that 43.8 % of packages with dependencies transitively reach a package known to have been malicious at some point. Install-time code execution is not an incidental flaw in npm; it is a design property of the resolver, and it is the single most predictive signal of malice currently known. This result confirms, at the scale of a registry and in probabilistic terms, what our earlier work established qualitatively: arbitrary code execution during dependency installation is the dominant attack vector in npm, and its availability differs sharply across package ecosystems [33].

Vulnerability tracking has a structural blind spot. Scanners determine whether a project is affected by a known vulnerability by examining the project's history. Forking—creating a new repository that copies an existing project's code and history—breaks this assumption: a repository forked after a vulnerability is introduced but before it is fixed inherits the flaw but not the fix. We generalised the evaluation semantics of the OpenSSF Open Source Vulnerability format [43] from repository-local histories to the global commit graph of public code archived by Software Heritage, propagating the introduction and fix of vulnerabilities across shared development histories (i.e., forks) [3]. Starting from 7 162 repositories referenced in the OSV database, our analysis reaches 2.2 million forks spread over 312 distinct code-hosting platforms. Of the 158.9 million commits identified as presumed vulnerable, 86.3 million—54.3 %—exist *only* in fork histories and are therefore invisible to every repository-local scanner. A further 1.7 million forks carry an unpatched head commit. After filtering for real user bases and high severity, manual auditing confirmed 135 one-day vulnerabilities at a precision of 0.69, and maintainers independently confirmed nine high-severity cases, including forks of the Linux kernel maintained by hardware vendors.

The historical record itself is mutable. All of the above is computed over what code-hosting platforms make visible, and that record is neither complete nor stable. Distributed version control systems allow history to be rewritten and force-pushed, and platforms discard the previous state when this happens. Analysis of 111 million repositories archived at least twice by Software Heritage shows 1.22 million repositories containing altered histories and 8.7 million altered commits, with 11.4 % of alterations occurring on main or master branches [21]. Approximately 800 000 retroactive modifications to licence files were observed, some changing the licence outright; and 13 million file removals involved paths referring to

secrets such as private keys and certificates—an operation that is insufficient to re-secure the system since archived copies persist and unrotated keys remain valid. Software Heritage is the only platform where these alterations can be observed at all.

Taken together, these four results converge on one claim: what a system contains, what it executes, and what its history was cannot be read off a manifest. Each has to be established empirically, and the four measurements above show current tooling getting contents, execution, and history wrong in ways that are systematic rather than accidental. Compliance reporting, vulnerability tracking, and provenance all consume this record, so each inherits its errors.

2.1.3 Regulation makes the gap actionable

The European Cyber Resilience Act [25] converts SBOM generation from a voluntary security practice into a legal obligation, with provisions entering into force in September 2026 and full compliance required by December 2027. In practice this obligation will be discharged using the same open-source generators measured above. Our results show that this is not a neutral choice. Current SBOM standards [11, 13] specify in detail *how* a dependency should be represented once it has been selected, but they do not define *which* dependencies are in scope. Scope, provenance, and graph structure are left to each tool's discretion. The consequence is that *the choice of tool is itself a compliance variable*: a publisher who follows the letter of the regulation by producing an SBOM has no practical means of determining whether the SBOM is complete. This is not a matter of tools needing bug fixes. It is a missing specification, and supplying it is a scientific problem in formal semantics as much as an engineering one.

The same reasoning underlies the sovereignty argument. To audit the software running its critical infrastructure, Europe requires both infrastructure it controls and ground truth that does not route exclusively through non-European registries and vulnerability databases. Software Heritage, an Inria-rooted universal archive of source code, is that infrastructure; making it usable as a security ground truth is a substantial part of this project.

2.1.4 The user-facing edge of the same problem

The opacity that hides a vulnerable transitive dependency also hides a data exfiltration path and a tracking vector. A third-party `<script>` element loaded by a web page and an installation script executed by a package manager are the same architectural failure—ambient authority granted to code of unverified provenance—observed at run time and at install time respectively. In both cases the host grants full access to its resources on the basis of a name.

Device fingerprinting is the limiting case of this problem, and it is structurally different from every other defect class considered in this proposal. There is no injected malicious code and no vulnerable dependency. The attack surface consists entirely of *legitimate platform interfaces*, exposed deliberately by the platform's own developers, whose aggregate identifiability was never a design consideration. A browser or mobile operating system exposes thousands of small, individually innocuous properties—rendering behaviour, available fonts, audio processing characteristics, hardware capabilities. Their combination, however, identifies a device with high probability: what is innocuous in isolation is decisive in aggregate. The Android Open Source Project exposes on the order of 250 000 methods and attributes to an application holding no permissions at all [1]; Chromium exposes some 16 000 interface elements to any webpage you visit[6]. Neither surface has ever been audited from the source code that produces it. The research community discovers fingerprinting vectors reactively, by measuring browsers and scripts already deployed in the field, years after the code that introduced them was written and reviewed.

2.2 General objective

> **CANCUN's objective is to make the composition of large-scale open-source software auditable: to establish, with quantified evidence, what a piece of software is actually made of, and to find what is wrong with it, at the scale of the global open-source ecosystem.**

This decomposes into two objectives, one per research axis.

GO1 — Establish composition (Axis 1). Build evidence-based, formally grounded models of what a software system is composed of, spanning all reuse mechanisms rather than declared dependencies alone; and use those models to specify package management and distribution architectures that are secure by construction rather than by retrofit.

GO2 — Find what is wrong (Axis 2). Detect, diagnose, and triage security and privacy defects across that composition at ecosystem scale—known vulnerabilities propagating through undeclared reuse, privacy defects and fingerprinting vectors in platform interfaces, and the structural health signals that determine which components warrant scrutiny.

Two points about the structure of the project deserve to be stated explicitly, since they are the first questions a reader is entitled to ask of a two-axis proposal.

The axes are coupled, not parallel. Axis 1 determines *what* is in a system and *how it got there*; Axis 2 determines *what is wrong with it* and *how much that matters*. Provenance classification flows from Axis 1 to Axis 2: when Axis 1 establishes that a directory in a project is a vendored copy of a library, or that two repositories share a development history, Axis 2 can propagate vulnerability information along that newly visible edge. Evidence flows back in the other direction: the distribution of defects that Axis 2 observes in the field tells Axis 1 which forms of reuse its models must capture, and which risks its architectures must foreclose. The two axes operate over one shared foundation—the global commit graph, registry data, and build observations—and neither is viable alone.

2.3 Scientific and technical challenges

We identify six challenges. Each is addressed by a named part of the research programme, and each part of the research programme is motivated by a named challenge; Table 2 gives the mapping.

	Challenge	Addressed in
C1	Provenance and visibility across heterogeneous reuse	§4.1.1, §4.2.1, §4.2.2
C2	Scale versus precision	§4.1.1, §4.2.1, §4.2.2
C3	Soundness of the historical foundation	§4.1.1, §4.2.1
C4	Policy inference under retrofit constraints	§4.1.2
C5	Predictive validity of ecosystem risk indicators	§4.2.3
C6	Agentic contributions and the erosion of authorship as a provenance signal	§4.1.1, §4.1.2, §4.2.1, §4.2.3

Table 2: Traceability between scientific challenges and the research programme.

C1 — The provenance and visibility gap

No single representation covers the nine classes of reuse in Table 1. Four sub-problems are open. Identifying a component from its *content* rather than its declaration, when metadata has been stripped by copying. Approximating statically the set of modules that a program may load dynamically. Tracing execution and data flow across language boundaries, where a program written in a high-level language calls into compiled code through a foreign function interface. And the complete absence of any provenance model for code synthesised by large language models, which reproduces patterns learned from open-source corpora without retaining their origin or licence.

We state this challenge in a more general form than is customary, because the generalisation is one of the project's contributions. The problem is not specific to dependency manifests. *Wherever software declares a contract, the declaration diverges from behaviour, and the divergence is unmeasured.* A package manifest declares dependencies; an OpenAPI specification declares the interface a service exposes; an Android manifest declares the permissions an application requests. In each case the declaration is the artefact that tooling, auditors, and regulators consume, and in each case its relationship to actual behaviour is assumed rather than established. That tools disagree even on the *declared* subset of dependencies—the

easiest case, where a machine-readable answer exists [4]—indicates how far the field is from handling the undeclared ones.

Stochastic reuse sharpens this challenge rather than expand its scope. A fragment generated by a language model may reproduce a vulnerable function from a library the model encountered during training, with no dependency declaration, commit history, or metadata linking the two. The provenance model must therefore accommodate edges that are *inferred* rather than observed: the connection between generated code and its training-origin library is a probabilistic attribution, not necessarily a declared or copied link. Making such attributions measurable is the subject of §4.1.1 (c).

C2 — Scale versus precision

The open-source ecosystem expands along two orthogonal dimensions: space, with hundreds of millions of repositories, and time, with billions of commits. Analyses that are precise are not tractable at this scale, and analyses that are tractable are not precise. The conflict can be quantified. Established techniques for identifying which versions of a library are affected by a vulnerability take between 0.01 and 187 seconds per library version; applying them to the 53 million pairs of fork and vulnerability that our global propagation identifies would require between 2.4 years and 30 000 years of computation, and that estimate assumes all forks have already been identified and are locally available, which is itself infeasible [3]. Progress requires representations that change the complexity class rather than faster implementations of the same algorithms: global deduplicated graphs of public code [46], and structural sharing across versions that shifts historical analysis from being linear in the number of versions to linear in the amount of unique code [10, 27].

C3 — Soundness of the historical foundation

Every provenance claim, vulnerability range, and SBOM in current practice is computed over the state that code-hosting platforms make visible. That state is incomplete—vulnerable forks are spread across 312 platforms, of which one dominates public attention—and it is mutable, since histories are rewritten and prior states are discarded [21]. A model built on an unsound foundation inherits an unquantified error. Two research questions follow. What is the magnitude of that error, and how should provenance models express confidence rather than assert certainty? And, more interestingly, *is tampering itself a signal*? A project that retroactively rewrites its release history, silently changes its licence, or purges secrets without rotating them is exhibiting a behaviour that no vulnerability database records and no scanner detects.

C4 — Policy inference under retrofit constraints

Restricting what an installed package is permitted to do is a solved *mechanism*: capability-based sandboxing and per-package permission systems have been demonstrated repeatedly [41, 15]. What is not solved is *authoring the policies*. No one will hand-write permission manifests for 3.5 million packages. A policy that over-approximates what a package needs grants back the authority it was meant to remove; a policy that under-approximates breaks the installation. The challenge is to infer minimal capability sets automatically, from static analysis and observed behaviour, with the soundness and compatibility trade-off quantified rather than assumed—and to do so on legacy ecosystems that cannot be redesigned. This challenge was identified as future work by the doctoral thesis on supply chain attacks conducted in this laboratory [26], which called for a secure-by-design package management system; CANCUN takes it up directly.

C5 — Predictive validity of ecosystem risk indicators

Deep auditing does not scale to millions of components, so something must decide where effort is spent. Existing health and risk indicators [12] encode expert judgement and are, to the best of our knowledge, unvalidated against outcomes: no published evidence establishes that a low score predicts compromise or abandonment. The obstacle is methodological rather than conceptual. The events to be predicted are rare, severely class-imbalanced, and labelled years after the fact. A closely related problem, stated explicitly as open by our own work, is that *no ground truth exists for vulnerability propagation across reuse*

mechanisms: our fork analysis had to treat maintainer responses as the only available oracle, which permits measuring precision but not recall [3]. The same scarcity affects malicious package detection, where labelled samples are held privately by vendors and removed from registries once identified [26, 31].

C6 — Agentic contributions and the erosion of authorship as a provenance signal

Autonomous coding agents now generate and submit pull requests to open-source repositories. This creates a class of supply chain risk that is neither a dependency vulnerability nor a malicious package: code of unknown training provenance, attributed to an agent identity rather than a human author, reviewed and merged under trust assumptions designed for human contributions. The risk is structural rather than adversarial. A maintainer reviewing an agent-generated contribution has no mechanism to assess whether it reproduces a vulnerability from the model's training corpus, introduces a hallucinated dependency [22], or carries patterns incompatible with the project's security posture. Threat modelling for agentic systems identifies memory poisoning, tool misuse, and privilege compromise as primary risks [19]. Seen from the supply chain, the additional risk is that the chain itself becomes the medium through which agentic outputs propagate. CANCUN addresses this challenge by extending its composition and propagation models to machine-generated code (§4.1.1, §4.2.1) and by applying capability inference (§4.1.2) to the question of what an autonomous agent should be permitted to do within a repository.

2.4 Research method

CANCUN follows a four-stage empirical protocol. We state it concretely rather than in the abstract, because all three of the preliminary results summarised in §2.1.2 instantiate it, which makes the claim verifiable rather than aspirational.

1. **Measure at ecosystem scale before modelling.** We characterise actual practice over a complete registry or the global archive before proposing any formalism. Our preliminary studies operate on 3.56 million packages [14], five billion commits [3], and 3 326 projects across two ecosystems [4].

2. **Construct independent ground truth.** We never benchmark tools against one another, because agreement between tools is not evidence of correctness. For the SBOM study we regenerated lock files from scratch and wrote parsers for nine lock file format versions, rather than trusting the lock files committed by developers or the consensus of the tools under test [4].

3. **Validate through responsible disclosure.** For defect-finding work, the maintainer of the affected project is the only real oracle. Our fork analysis proceeded from automated propagation, to manual audit by the authors (precision 0.69), to notification of every affected maintainer, yielding nine independently confirmed high-severity vulnerabilities [3]. We commit to extending this protocol to the other two threads: disclosure to SBOM tool maintainers for the divergences we catalogue, and to platform vendors and extension authors for privacy and fingerprinting findings.

4. **Release the corpus, not only the tool.** Findings of this kind have a limited shelf life. The same SBOM generators produced *empty* inventories for a common lock file format two minor versions before our study and *partial* ones during it [4]. A result reported as a number decays; the same result encoded as an executable test case does not. Two of our planned deliverables are therefore corpora rather than prototypes: a conformance benchmark for SBOM generation, and a retrospective cohort for validating risk indicators.

Contributions are validated empirically through a combination of quantitative and qualitative evaluation: large-scale measurement, controlled experiments, manual audit, maintainer confirmation, and—where practitioners are the subject—structured interviews and in-vivo assessment.

3 Application domains

The methods developed in CANCUN are generic. We validate them in three domains, chosen because each exercises a distinct property of the approach and because the team has established access to the necessary data and partners.

3.1 Package ecosystems and software forges

The npm, PyPI, crates.io and Maven registries, Linux distributions, and the 300-odd code-hosting platforms reachable through Software Heritage constitute the primary validation ground for both axes. This is where scale is real: millions of packages, hundreds of millions of repositories, billions of commits. It is also where all three of our preliminary results are situated, and where the regulatory pressure of the Cyber Resilience Act will be felt first, since these registries are how European software manufacturers obtain the components they must now inventory.

3.2 Web and mobile platforms

Web browsers and mobile operating systems are the universal runtimes of the digital age, and they play two distinct roles in this project. As *artefacts*, Chromium and the Android Open Source Project are among the largest polyglot codebases in existence, with deep dependency trees, extensive build-time configuration, and vast third-party ecosystems of extensions and applications; they are therefore a demanding validation target for Axis 1's composition analysis. As *platforms*, their programming interfaces are the foundation for Axis 2's privacy work: it is the interface surface itself, not any malicious component within it, that creates the identifiability risk. This domain subsumes the application-level scenarios that motivate privacy enforcement—digital skimming attacks on e-commerce sites, unauthorised tracking scripts on services handling personal data—which are instances of the same third-party trust problem rather than separate domains.

3.3 Regulated and sovereign infrastructure

Systems subject to Cyber Resilience Act obligations or to sovereignty constraints—public administration, critical infrastructure, and defence—need capabilities that do not currently exist: the ability to vet and freeze a trusted subset of open-source components, to validate independently the inventories that suppliers provide, and to conduct all of this on infrastructure that Europe controls. Our partnership with ANSSI and our work with Software Heritage target exactly these capabilities. The relevant scientific content is the same as in the other two domains; what differs is the requirement that results be defensible to an auditor rather than merely useful to a developer.

4 Research program

4.1 Axis 1: Software Supply Chain Security

> **Objective.** Define what a software system is composed of, on the basis of evidence rather than declaration, and specify distribution architectures in which composition is trustworthy by construction.

Work on software composition currently conflates two directions that pull in opposite ways, and separating them is the organising idea of this axis. The *bottom-up* direction asks what a given artefact is actually made of. It is descriptive and empirical, it must accept legacy ecosystems as they are, and its output is a model of observed practice. The *top-down* direction asks what a trustworthy composition mechanism would look like. It is normative and design-oriented, and it is constrained by what can be retrofitted onto infrastructure that cannot be replaced. Neither suffices alone. Bottom-up work without a normative anchor produces exactly the situation we measure today, in which three standard-conforming

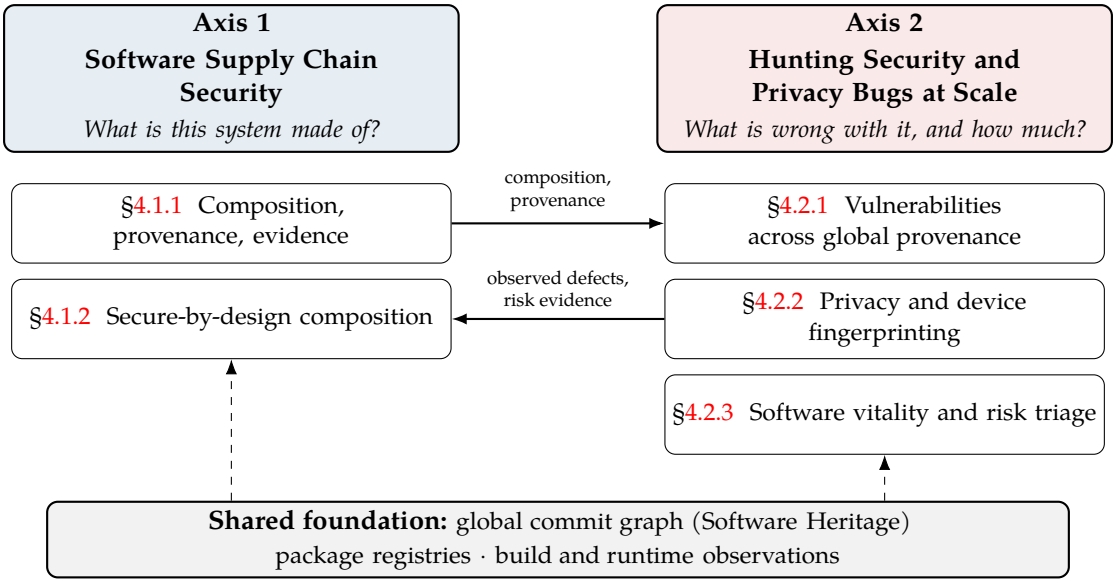

Figure 1: The CANCUN research programme: two coupled axes over one shared foundation. Axis 1 establishes what a system is composed of and how it should be composed; Axis 2 finds and prioritises what is wrong with it. Provenance flows left to right; evidence about observed defects flows right to left.

tools return three different answers for the same project. Top-down work without empirical grounding produces mechanisms that nobody adopts. *Formalisation belongs at the junction*: the formal semantics must be rich enough to describe what ecosystems actually do, and precise enough to serve as a specification for what they should do.

4.1.1 Bottom-up: composition, provenance, and evidence

Addresses C1, C2, C3, C6.

Context: a taxonomy of reuse. Modern software reuses code through a spectrum of mechanisms, each with different implications for auditing. *Managed dependencies* are declared in a manifest and resolved by a package manager, and are the only form that current tooling handles well. *Forking* clones an entire repository to create a divergent line of development. *Vendoring* copies third-party source or binaries directly into the host project's tree, usually stripping the metadata that would identify them. *In-line reuse* copies individual functions or fragments. *Dynamic loading* resolves modules at run time based on configuration or program state. And *stochastic reuse* integrates code synthesised by a language model from patterns learned across open-source corpora, reproducing logic—and vulnerabilities—without any record of origin. Only the first is visible to standard inventory tools; the remaining five constitute *shadow dependencies* that propagate vulnerabilities without appearing in any inventory.

(a) Formal semantics of composition and scope. We model the ecosystem as a directed graph $G = (V, E)$ in which vertices are artefacts (specific versions of components) and edges carry a type: strict, peer, optional, development, build, workspace, or dynamic. Resolution—the process by which a package manager turns a set of version constraints into a concrete dependency tree—is formalised as a constraint satisfaction problem, one per ecosystem. This per-ecosystem treatment is the point: npm's nested trees, Maven's nearest-wins arbitration, and Cargo's version unification are *different semantics*, not different implementations of one semantics, and treating them uniformly is the root of much current tool divergence.

The open problem, sharpened by our measurements, is *scope* rather than representation. Standards define how to encode a component once it has been selected and are silent on selection itself. We will define a canonical scope and provenance semantics in which every component in an inventory is annotated

with two facts: *why* it is present (declared in a manifest, resolved by a solver, inferred by a heuristic, or observed during a build) and *for which lifecycle phase* it is relevant. The second annotation resolves a genuine disagreement in the literature. Build and test dependencies are false positives for an inventory describing what a deployed application contains, and they are legitimate, security-relevant components for an inventory describing what a contributor's machine and continuous integration pipeline execute—and build tooling is itself a documented attack target [39]. Rather than adjudicate, we make the distinction explicit: a development inventory and a deployment inventory are complementary artefacts, and filtering between them must be an auditable step rather than an undocumented tool default.

(b) The declaration–reality gap, generalised. As argued under C1, package manifests are one instance of a general phenomenon. Interface contracts are a second instance, and empirically a worse one: more than 70 % of developers report that web API documentation is missing, incomplete, incorrect, or outdated, and a 2022 breach exposing data from 5.4 million users of a major social platform originated in exactly such a divergence between a requirement, a change to code, and the interface it exposed. Permission declarations on mobile platforms are a third instance, and connect directly to §4.2.2.

The ANR JCJC project PEEPS, coordinated by S. Challita and running from 2026 to 2030, supplies the technique on the interface side: an enhanced metamodel for the OpenAPI standard, together with static and dynamic analysers that reconstruct an interface model from code and tests rather than from documentation, and co-evolution machinery that keeps declared artefacts aligned as code changes. CANCUN's contribution is the unification—a common formal treatment of *declared artefact versus observed behaviour* covering manifests, interface contracts, permission declarations, and agent-submitted contributions, with the divergence measured rather than assumed. We state the boundary plainly: requirements engineering and stakeholder-facing specification languages remain objectives of PEEPS and are not pursued by CANCUN.

A fourth instance has emerged with LLM-based coding assistants and autonomous agents: the gap between the declared intent of a contribution and the provenance of the code it contains. A pull request submitted by a coding agent declares a human-readable intent—"fix the authentication bug in module X"—but the code may integrate patterns from multiple training sources, some carrying vulnerabilities; the declaration, commit message and description, is decoupled from the provenance of the generated code. This is not hypothetical: agentic coding tools moved from experiment to practice in 2025, and early empirical studies of their pull requests report modification patterns and issue rates that differ measurably from human contributions. Our composition model therefore treats machine-generated code as a first-class provenance class rather than a degenerate case of vendoring, because the attribution problem is fundamentally different: there is no source commit to trace, only a model and a prompt.

(c) Evidence-based generation, and identification as an entropy problem. Detecting vendored code requires recognising a library from its content when its name and version have been removed; we will develop techniques based on hashing of abstract syntax trees and on fuzzy hashing that tolerates local modification. Dynamic imports require static approximation of the set of modules a program may load. Build-time instrumentation, in which plugins for common build systems record what was actually fetched and linked, provides ground truth against which declaration-based inventories can be measured.

We propose to frame component identification as an *entropy problem*, which both sharpens it and connects the two axes. The question is not "does this code match a known library" with an arbitrary similarity threshold, but: how many bits of structural signal are required to identify a stripped component uniquely, and what is the residual set of candidates that remain indistinguishable? This is formally the same question as device fingerprinting [49]—how many observable attributes are needed to single out one device among many—applied to code rather than to devices, and it replaces heuristic thresholds with a measurable notion of identifiability. The team's expertise in the fingerprinting formalism is directly transferable here, and the transfer runs in both directions.

The framework extends directly to stochastic reuse. A fragment generated by a language model carries structural signal inherited from its training origin: function signatures, control-flow shapes, error-handling idioms. Attributing the fragment to a specific library in the training corpus then reduces to the same question as before: how many bits of that signal are needed to single out one origin among

many? The analogy with device fingerprinting is interesting—the model's output distribution plays the role of the device, and the training corpus plays the role of the population of devices—so the same techniques apply: AST hashing, fuzzy hashing, and information-theoretic bounds on identifiability, only with weaker signals and larger candidate sets. Stochastic reuse is therefore the stress test of the identification framework. Generated code is the setting in which provenance signal is weakest; whatever identifiability the techniques achieve there is a floor for what they deliver on the easier classes, from vendored directory trees, where whole files survive the copy, up to verbatim copies, where content hashes match exactly.

(d) The published-artefact gap. There is a question that the entire inventory ecosystem assumes away and that nobody has measured at scale: what is the relationship between the artefact a consumer installs and the source code an analyser examines? These are different objects, joined by a build process that is generally not verified. All three of the SBOM generators we studied [4] analyse source repositories, while consumers install published archives. The Reproducible Builds effort [36] makes the relationship verifiable for projects that participate; no one has measured how often it fails across ecosystems for those that do not.

The best-known instance of divergence is the XZ Utils backdoor, in which malicious content was present in the released archive and absent from the publicly visible repository [29, 23]. We are deliberately not building this research direction on a single incident. The general phenomenon is mundane and frequent: files present in a published archive but absent from the repository, content injected by build steps, packaging directives that include or exclude files silently, and prebuilt binaries with no corresponding source at all. We will conduct a cross-ecosystem measurement of divergence between published artefacts and archived source, using Software Heritage for the source side and registry mirrors for the artefact side, and produce a taxonomy of causes distinguishing benign build output and packaging convention from the unexplained residue. This measurement is possible only for a team with access to a universal source archive.

(e) Soundness under a mutable foundation. We will quantify the error that history rewriting and platform data loss introduce into provenance claims, and express the result as a soundness precondition: a provenance model is only as sound as the completeness and immutability of the graph it is computed over. The deliverable is a provenance model that carries confidence rather than asserting certainty, together with empirical bounds on that confidence derived from the observed rate and distribution of history alterations.

(f) A conformance benchmark for SBOM generation. Each divergence case catalogued in our tool study—package aliasing, peer-dependency suffixes, symbolic link entries, workspace placeholders, monorepo path prefixes, absent lock files, and the three incompatible graph topologies the tools produce [4]—becomes a minimal project with a specified expected output. The resulting suite lets any generator, present or future, be checked against each edge case individually. This positions CANCUN and Software Heritage as a neutral validation authority rather than as the authors of a fourth competing generator, and it directly addresses the shelf-life problem identified in our research method.

Four-year perspective. Items §4.1.1 (b), (d), (e) and (f) are the priority of the first four years, since each is anchored in a measurement we have already published or can begin immediately. The formal semantics (a) and the conformance benchmark (f) consolidate their results and are scheduled for a second phase.

Ten-year perspective. A canonical formal semantics for software inventories, with verified translation between the major standards, adopted as the reference model that conformance tooling targets—ending the situation in which the choice of tool determines the answer.

4.1.2 Top-down: secure-by-design composition

Addresses C4, C6.

Context. Package managers were designed for an era of benevolent collaboration. They grant *ambient authority*: an installed package obtains the full privileges of the user who installed it, including access to the file system, the network, and environment variables holding credentials. Install-time hooks turn what should be the mechanical act of resolving and unpacking dependencies into arbitrary code execution.

(a) Empirical characterisation of the trust flaw. The measurements reported in §2.1.2 establish the magnitude of the problem for npm. We will extend this characterisation across ecosystems—PyPI, crates.io, Maven, and Go—using the attack taxonomy developed in this laboratory as the frame of reference. That taxonomy systematises 107 distinct attack vectors against open-source supply chains, links them to 94 documented real-world incidents, and maps them to 33 mitigating safeguards whose utility and cost were assessed by expert and developer surveys [7, 32]. Its companion study establishes how third-party dependencies actually achieve execution on downstream systems across seven ecosystems [33]. Go is the natural control case, since it provides no installation script mechanism at all: comparing attack technique distributions between ecosystems that permit install-time execution and those that do not isolates the effect of the mechanism from the effect of ecosystem popularity.

(b) Longitudinal measurement of an ecosystem-wide intervention. The npm ecosystem has recently disabled the execution of installation lifecycle scripts by default. This is a rare opportunity: an intervention applied to an entire ecosystem at a known date, for which we hold a detailed pre-intervention baseline [14]. We will measure, without committing in advance to a directional prediction, how the distribution of attack techniques observed in confirmed malicious packages changes across the transition; whether developers adopt the new default, opt out of it, or work around it; and whether the minority of legitimate uses of installation scripts contracts or migrates to other lifecycle phases. Whatever the outcome, the study establishes a reusable methodology for evaluating ecosystem-wide security interventions—of which the Cyber Resilience Act will produce many.

(c) Capability inference. This is the axis's core scientific contribution, and it is deliberately not "build a sandbox". Sandboxing mechanisms exist. What does not exist is a way to obtain, for each of millions of packages, a policy describing the minimal set of capabilities it legitimately requires: which file system paths, which network endpoints, which environment variables, which subprocesses. We will infer such policies from a combination of static analysis and observed installation and build traces, and—this is the scientific content—characterise the resulting trade-off surface. How much over-approximation is required to retain installation compatibility for $n\,\%$ of a real registry? What residual authority does that over-approximation leave to an attacker? Policies will be evaluated by their coverage of documented attack techniques from the taxonomy [7], rather than against intuition about what packages ought to be allowed to do.

The framework extends to autonomous coding agents. An agent with write access to a repository holds the full privileges of a contributor: it can add dependencies, modify build scripts, introduce install-time hooks, and alter permission manifests. Asking what minimal capabilities an agent needs for its task is the same inference problem as for a package, with one additional dimension: agent behaviour is non-deterministic and prompt-dependent, so a policy must cover the *distribution* of actions the agent may take, not the actions observed in a single invocation. The least-privilege mechanisms developed here for package management therefore transfer to agent governance, where excessive autonomy and tool misuse are recognised as primary risks [19]; the TAP collaboration (§5), which studies vulnerabilities in LLM-generated code, provides a controlled setting for initial evaluation.

(d) From properties to compliance. We will map the obligations imposed by the Cyber Resilience Act onto properties that can be checked mechanically against the model of §4.1.1: completeness of provenance, explicit declaration of scope, and traceability of vulnerability disclosure. This work is deliberately narrow, covering security-by-design and vulnerability-management obligations. We do not attempt to formalise the semantics of data-protection law, which requires legal expertise the team does not claim.

Four-year perspective. Items §4.1.2 (a) and (b) will be addressed in the first four years; capability inference (c) starts as soon as their installation and build traces provide the data it requires, and the compliance mapping (d) follows in a second phase.

Ten-year perspective. A least-privilege distribution mechanism piloted with a real registry or a sovereign infrastructure operator, consuming the vitality signals of §4.2.3 at resolution time so that capability restriction and risk assessment reinforce one another.

4.2 Axis 2: Hunting Security and Privacy Bugs at Scale

> **Objective.** Detect, diagnose, and prioritise security and privacy defects across the whole composition surface—including the reuse paths that Axis 1 makes visible for the first time—at the scale of the global open-source ecosystem.

Three families of defect are in scope, and they share a foundation rather than a technique. Known vulnerabilities propagate through reuse mechanisms that current tooling cannot see (§4.2.1). Privacy defects and device fingerprinting vectors arise from platform interfaces that no one has audited from source (§4.2.2). And because auditing cannot be applied everywhere, structural health signals must decide where it is applied (§4.2.3). What unifies them is that each operates over the global provenance graph rather than over individual repositories.

4.2.1 Known vulnerabilities across global provenance

Addresses C1, C2, C3, C6.

(a) Global history analysis. Our established base is the generalisation of vulnerability evaluation semantics from repository-local histories to the global commit graph [3], whose results are summarised in §2.1.2. Rather than present those results as finished, we take the limitations that the study itself identifies as the research programme.

Equivalent-patch detection is the dominant source of false positives. Maintainers frequently fix a vulnerability by means that automated propagation cannot recognise—a rewritten fix, a backport applied by hand, a change that removes the vulnerable code path rather than repairing it. Detecting fixes that were cherry-picked without leaving a commit message trace, using content-based patch identifiers, raises precision from 0.41 to 0.69. Recognising *semantically* equivalent patches is the open problem beyond that, and it requires program analysis rather than history analysis.

Reachability after divergence sets the precision ceiling. A fork that has diverged substantially from its upstream may contain the vulnerable code without the vulnerable path being reachable. Our current filter is a conservative heuristic—if a file modified by the fix is absent from the fork, the pair is discarded—which removes 43 % of candidate pairs and still cannot establish that the remaining ones are exploitable. Proving applicability in a diverged codebase is the substantive problem.

Recall is currently unmeasurable, because no ground truth exists for vulnerability propagation across reuse mechanisms. Constructing one is stated here as an objective of the project, not as a caveat on a result.

(b) Beyond forks: propagation across all reuse classes. The propagation method rests on commit sharing, which covers forking. We will extend the same semantics to vendored trees, copied fragments, and code mediated by language models, using the identification techniques of §4.1.1 to establish the edges along which information propagates. This is the point of tightest coupling between the two axes: Axis 1 supplies the edge, Axis 2 propagates the label along it.

The LLM-mediated case is the hardest and the most consequential. A vulnerability in a library that was part of a model's training corpus may be reproduced, partially or fully, in code generated by that model, in a project with no declared dependency on the library and no commit history linking the two. Propagating vulnerability labels along this inferred edge requires three steps: identifying that a generated

fragment is structurally attributable to a specific library version (§4.1.1 (c)); determining whether the vulnerable code path survived generation; and assessing reachability in the host project. Each step is a research problem, and the third is the reachability-after-divergence problem already identified for forks in (a), compounded by the fact that the "fork" is implicit and probabilistic rather than declared and structural. We will proceed incrementally: first on synthetic cases where the training origin is known, using the TAP project's controlled generation pipeline, then on agent-generated pull requests mined from public repositories.

A specific attack vector connects this strand to the malicious-component work of §4.2.3 (d). Language models hallucinate package names that do not exist, and a substantial fraction of these names recur deterministically across runs; adversaries can therefore register them in advance with malicious payloads—*slopsquatting* [22]. The same mechanism generates import statements for packages that exist but are not declared, creating shadow dependencies invisible to any manifest-based tool. A composition model that already handles undeclared reuse through evidence-based identification rather than manifest lookup is the natural framework for detecting both.

(c) Integrity as a signal. Force-pushes on release branches, retroactive licence changes, removal of committed secrets without rotation, and anomalies in maintainer account activity are observable at global scale in an archive that preserves superseded states [21]. They serve two purposes here. They are red flags that feed the risk model of §4.2.3, and they constitute a defect class in their own right: a retroactively suppressed licence grant is a genuine supply chain harm for which no vulnerability identifier is ever issued and no scanner exists.

(d) Patch propagation at scale. Making historical analysis tractable requires representing an entire project history as a single structure with sharing between versions, so that a query is evaluated once and its result mapped to every version containing the shared structure [10, 8]. This shifts complexity from linear in the number of versions to linear in the amount of unique code, and enables two capabilities: locating the exact commit that introduced a vulnerability across a project's full history, and transplanting a fix into an older version when upgrading is blocked by interface changes. Impact-analysis techniques developed for artefact co-evolution, which avoid re-analysing an entire project at every commit, transfer directly to this setting.

Four-year perspective, and an explicit exclusion. Items §4.2.1 (a)–(c) are the priorities of the first four years; patch propagation (d) builds on their results. We deliberately exclude from this core the generative synthesis of patches for previously unknown vulnerabilities, together with the associated problem of distinguishing a plausible patch from a correct one. That territory is well covered by other groups [47, 38] and by the team's own TAP collaboration; claiming it would dilute a sharper and better-evidenced contribution.

Ten-year perspective. We aim to move from a lookup service to propagation integrated into continuous integration pipelines, with automated proposal of patches to affected maintainers.

4.2.2 Privacy and device fingerprinting

Addresses C1, C2.

A privacy defect is a defect class with the same machinery as a security vulnerability and a different sink: the harm is identifiability or exfiltration rather than memory corruption or privilege escalation. But device fingerprinting, as noted in the context, is structurally unlike every other defect class in this proposal, and saying so precisely is what makes it a research problem rather than an application of the preceding section: *there is no malicious code and no vulnerable dependency*. The surface consists of legitimate platform interfaces, exposed deliberately, whose aggregate identifiability was never considered at design time. Three strands of comparable weight follow.

(a) **Source-level provenance of the fingerprinting surface.** The state of the art discovers fingerprintable surface *reactively*, and treats the platform as a black box. Measurement platforms observe deployed browsers and devices to estimate how uniquely they can be identified [49, 48]; detection tools observe deployed scripts to determine which are extracting identifying information [42]. Neither traces identifiability back to the source code and development history that introduced it.

We propose the complementary direction, which requires two capabilities this team holds jointly and which we believe no other group combines: analysis of browser and operating system histories at scale, and calibrated corpora of real device fingerprints. Concretely: trace attribute-exposing interface surface through the commit histories of Chromium and the Android Open Source Project, establishing for each identifying attribute when it was introduced, by which change, and under which build configuration or feature flag; attribute a quantity of identifying information to individual code changes and configuration options, turning the qualitative statement "this platform is fingerprintable" into the measurable statement "this change added n bits"; and predict identifiability *before* release, inverting a model in which the consequences of new interface surface are discovered only after deployment and exploitation in the wild. The interface-model extraction techniques developed in PEEPS apply directly at this scale, where enumerating the surface is itself a research problem.

(b) **Hardware and low-level side channels.** Rendering and driver-derived attributes exposed through graphics interfaces, audio processing characteristics, timing channels, and enumeration of fonts and codecs each leak information about the underlying hardware and software stack. The emphasis here is on the two properties that make an attribute *trackable* rather than merely diverse: stability over time, and uniqueness across the population. Diversity without stability produces an attribute that changes between visits and cannot link them; stability without uniqueness produces one that is shared by millions of devices. Only the conjunction constitutes a privacy defect, and distinguishing the two is what separates a rigorous result from an alarming one. Recently stabilised graphics interfaces are the specific near-term opportunity: a large, directly hardware-exposed surface that has not been systematically measured.

(c) **Permission-gated identifiability and third-party script behaviour.** Our recent work established that an Android application holding no permissions at all can reach on the order of 250 000 methods and attributes, and can construct extensive device fingerprints from them [1]. The natural extension asks what *permissions purchase*: how much additional identifiability does each permission grant release, and are permissions priced correctly against the information they expose? This is the third instance of the declaration–reality gap of C1—a manifest declares which resources an application may access, and says nothing about the identifiability that access confers.

On the web side, we will quantify the information a script extracts from its environment at run time, rather than matching against signatures of known trackers, so that novel vectors are detectable rather than merely catalogued after the fact. Distinguishing legitimate from tracking use of the same interfaces is a dual-use classification problem—a rendering engine and a fingerprinting script issue the same calls—which we approach through the structure of execution rather than its content. Finally, this strand connects back to §4.1.2: a third-party script holds ambient authority over the page, its storage, and the network in exactly the way an installation script holds it over the host machine, and the same question of automatically inferred capability policies applies, with existing content restriction mechanisms as the baseline to improve upon.

Autonomous agents operating *within* applications—browser extensions generated by language models, mobile applications with generated components, agent-mediated web interactions—introduce a further instance of the declaration–reality gap: an agent's declared permissions describe what it is authorised to access, while its actual behaviour, conditioned on prompts and model state, may explore a different and broader surface. The information-flow techniques of (d) apply, but the analysis target shifts from static code to the interaction between a model and its runtime environment, which is dynamic and non-deterministic by construction.

(d) **Supporting technique: information flow for personal data.** Tracking the flow of personal data from the points where it enters an application to the points where it leaves requires analysis that remains

precise through closures, asynchronous callbacks, and event loops. We treat this as supporting technique rather than as a headline contribution, and we make no attempt to formalise legal notions of consent or purpose limitation.

Four-year perspective. Items §4.2.2 (a)–(c) are the priorities of the first four years, with the information-flow technique (d) developed as their needs dictate.

Ten-year perspective. Accounting for fingerprinting surface integrated into platform development processes, so that identifiability becomes a reviewable property of a proposed change rather than a discovery made years after deployment.

4.2.3 Software vitality and risk triage

Addresses C5, C6.

Context. Deep auditing does not scale to 3.5 million packages or 2.2 million forks; something must decide where effort goes. Current practice relies heavily on popularity, which is a poor proxy. Our registry-scale analysis found low-visibility utility packages with transitive reach comparable to household names, because reach in a dependency graph is determined by structural position rather than by fame [14].

(a) A model of software vitality. We will define *vitality* as a composite estimate over signals including the number and identifiability of maintainers, the cadence of review and release, historical time to fix reported vulnerabilities, centrality and transitive reach in the dependency graph, the lag with which a fork integrates upstream changes, and the integrity red flags of the preceding section. We are explicit that this is not a quality score. It is an estimate of the probability that a component will be compromised, abandoned, or slow to patch—a forward-looking risk estimate, not a judgement about craftsmanship.

(b) Validation is the scientific problem. Defining such a score is easy; establishing that it predicts anything is not, and the absence of that demonstration is what distinguishes this work from existing indicators [12]. We will construct retrospective cohorts of components that were compromised or abandoned over a decade, score them at a point in time using *only* information available at that point, and evaluate on precision at a realistic auditing budget—how many of the top k flagged components turned out to matter—rather than on aggregate measures that are misleading under severe class imbalance and delayed labelling. Benchmarking against published heuristics isolates what graph structure and provenance signals contribute beyond expert judgement. The cohort is released as a community resource, which addresses in part the scarcity of labelled data that has been repeatedly identified as an obstacle in this field [26].

(c) Operational triage. Vitality becomes the ranking function over the candidates produced by §4.2.1. Our current pipeline reduces 1.76 million presumed vulnerable forks to 86 worth auditing through a hand-tuned cascade of popularity, severity, staleness, and divergence heuristics [3]. Replacing that cascade with a validated model is a concrete and measurable improvement over the team's own baseline, which is the form of evaluation we prefer.

(d) Detection of malicious components, and its adversarial limits. Vitality estimates the risk that a component becomes a problem; complementary work detects components that already are one. Detection based on language-independent features—most prominently the presence of installation hooks—has been shown to work across ecosystems and to find previously unknown malicious packages in the field, at a rate of 58 discoveries over 31 292 packages analysed [31, 35]. The open problem is robustness: recent work demonstrates that functionality-preserving obfuscation evades current detectors at a rate above 85 %, and that adversarial training recovers only part of the lost accuracy [18]. We will treat detector

robustness under adaptive adversaries, and the construction of shared labelled corpora that makes such evaluation possible at all, as an explicit objective rather than an assumed capability.

Machine-generated contributions require the same treatment from the opposite direction: a pull request from a coding agent is neither malicious by intent nor vulnerable by inheritance, but its provenance cannot be audited by the methods that work for human contributions. The vitality model will therefore include, among its candidate indicators, the share of agent-generated contributions in a project's recent history and the review those contributions receive: a project that merges large volumes of unreviewed generated code is structurally similar to one that adopts unreviewed dependencies, and the same triage logic applies. Like every other indicator in the model, this one enters as a hypothesis for the validation protocol of (b), not as an assumed risk factor.

Four-year perspective. Items §4.2.3 (a) and (c) are the priorities of the first four years; the adversarial-robustness programme of (d) proceeds in parallel through the Artishau collaboration.

Ten-year perspective. Vitality surfaced at dependency resolution time within the least-privilege mechanism of §4.1.2, and contributed back to Software Heritage as archive metadata available to the whole community.

5 Contracts

5.1 European initiatives

HiPEAC — High Performance, Edge And Cloud computing (EU CSA, 70 k€ for the team). Duration 12/2022–07/2026. Coordinator: Koen De Bosschere, UGent. Partners include Inria (PI: Olivier Zendra), Eclipse Foundation Europe, INSIDE, Universiteit Gent, RWTH Aachen, CEA, SINTEF, IDCI Italia, Thales, Cloudferro, and Barcelona Supercomputing Center. HiPEAC coordinates European research, development and deployment of computing systems technologies, producing roadmaps for next-generation infrastructures and supporting European digital autonomy across the computing continuum from cloud to edge and the Internet of Things.

5.2 International initiatives

TAP — Trustworthy Automatic Programming (DGA AID, 458 k€). Coordinators: Blaise Genest, Olivier Zendra. Partners: CNRS (IPAL / IRISA), National University of Singapore. 2024–2028. TAP identifies vulnerabilities in code generated by large language models, analyses and classifies them, and determines whether certain vulnerability classes are more frequent in generated code than in human-written code. Its objectives include automatic correction of vulnerabilities in generated code and hardening of the models themselves.

5.3 National initiatives

Software Heritage Sec (PTTC Campus Cyber, 480 k€). Coordination: Inria, with Olivier Barais and Stefano Zacchiroli as scientific leads. Partners: Inria, IMT, CEA, Sorbonne Université. 2023–2027. By analysing the evolution of source code over time at archive scale, the project aims to identify security risks early and to develop proactive mitigation. This contract directly funds the global history analysis reported in §2.1.2.

PEEPS — Preciseness in REST APIs (ANR JCJC, 345 793 €). Coordinator: Stéphanie Challita (Université de Rennes, IRISA). Partner: Luxembourg Institute of Science and Technology (J. Cabot). 09/2026–08/2030. Team involvement: Challita 36 person-months, Barais 8, Combemale 6, Khelladi 6, plus a doctoral student and a postdoctoral researcher. PEEPS develops a model-driven framework for defining, maintaining, and evolving precise REST interfaces, ensuring coherence between requirements, code, and documentation. It contributes to CANCUN the formal treatment of declared interface contracts against observed behaviour

(§4.1.1) and the interface-model extraction techniques applied to large platform surfaces (§4.2.2), and it draws its own corpus from Software Heritage.

Taranis (PEPR Cloud, 380 k€). Coordination: Inria. Partners: Inria, CNRS, Université de Rennes. 2023–2030. Taranis proposes abstractions of application structure that permit automated management of applications and infrastructures, allowing global multi-criteria optimisation of resource use.

WestOps (ANR, 350 k€). Coordinator: KEREVAL. Partners: KEREVAL, Ouest-France, Université de Rennes. 2025–2028. WestOps strengthens the robustness and trustworthiness of machine learning systems in production, addressing regression testing and robustness evaluation, rapid detection of performance deviation, and automated feedback loops for retraining under drift.

Privacy Bugs (Brittany region, 150 k€). Coordinator: Walter Rudametkin. Partners: DGA MI, Université de Rennes. 2023–2026. Privacy Bugs investigates tools and methods to detect and quantify privacy defects in front-end web applications, and funds the doctoral work of S. Roux.

5.3.1 Bilateral contracts with industry

CANCUN maintains recurring CIFRE conventions and bilateral contracts with close industrial partners.

Orange (CIFRE, 150 k€). Partners: Université de Rennes and Orange. 2025–2028. Patterns in software bills of materials; doctoral work of S. Plessis.

6 Impact and strategic positioning

The ambition of CANCUN extends beyond publication. Our goal is to improve the resilience of the software commons structurally, and we position our work as a form of public health for software: diagnostics that establish what a system contains and what is wrong with it, and treatments that can be applied at population scale.

6.1 Strategic partnership with Software Heritage

Software Heritage is the universal archive of source code, initiated by Inria. It is common to describe such an archive as critical infrastructure; we prefer to demonstrate the claim. Three of the results this proposal rests on are not merely *easier* with Software Heritage—they are *impossible without it*. Propagating vulnerability information across 2.2 million forks spread over 312 code-hosting platforms requires a deduplicated cross-platform commit graph that no individual platform provides [3, 46]. Measuring the rewriting of public development histories requires an archive that preserves superseded states, which platforms by construction do not [21]. Establishing a reproducible corpus for inventory tool comparison requires content-addressed identifiers for exact repository states [4]. This is a considerably stronger sovereignty argument than an abstract appeal to European infrastructure: there is a class of security questions that can currently be answered in Europe and nowhere else, and the capability is worth developing deliberately.

The partnership is bidirectional. CANCUN will contribute computed metadata back to the archive—provenance annotations, vulnerability labels on the commit graph, integrity assessments, and vitality estimates—transforming a preservation infrastructure into an active security observatory, and giving European institutions an autonomous capability to audit the supply chains of critical infrastructure.

6.2 Impact on the digital commons

CANCUN adopts an action-research methodology: in software engineering, the tool is a substantial part of the proof. We report one finding from our disclosure campaign because it shapes the project's strategy. Having notified the maintainers of every fork we confirmed as vulnerable, we received two

kinds of response. A minority of well-resourced projects fixed the vulnerability within days. The majority acknowledged the problem and explained that they lacked the resources to address it [3]. The bottleneck in open-source security is not awareness; it is maintainer capacity. That observation is the justification for automating remediation rather than merely improving detection, and it connects this section directly to the patch propagation work of §4.2.1. Our impact strategy is accordingly to deliver zero-friction tooling—notifications that arrive where maintainers already work, and patches that arrive ready to merge—rather than reports that add to their workload.

This bottleneck is compounded by autonomous coding agents: maintainers are now asked to review pull requests generated by systems whose training provenance they cannot assess, alongside the vulnerabilities they already lack the resources to fix. The same strategy answers both: a provenance annotation that flags a pull request as containing code attributable to a known-vulnerable library, or a capability analysis showing that an agent's proposed changes exceed the minimal scope of the stated task, reduces the reviewing burden instead of adding to it.

6.3 Societal impact

As open-source software pervades the daily lives of citizens through browsers, mobile applications, and connected devices, the privacy work of §4.2.2 has direct societal consequence. We will provide data protection authorities and non-governmental organisations with open auditing capability for detecting tracking and fingerprinting at scale, and provide platform developers with the means to assess the identifiability consequences of their changes before those changes ship. The public-facing measurement platforms the team operates additionally serve an educational role, allowing citizens to observe directly how identifiable their own devices are.

7 Transfer and software development

7.1 Transfer

CANCUN transfers results through CIFRE conventions and bilateral contracts with industrial partners (§5), and through direct engagement with the institutions responsible for national cybersecurity. Our work with ANSSI targets sovereign auditing capability; our work with Software Heritage targets the infrastructure on which that capability rests. Responsible disclosure is not only an ethical obligation but our primary validation instrument, and it constitutes transfer in its own right: every confirmed finding is delivered to the maintainers who can act on it.

7.2 Software development

HyperAST. A representation of an entire project history as a directed acyclic graph with structural sharing between versions, built by leveraging Git and incremental parsing, with a reimplementation of tree differencing over that structure and a reference solver indexed for efficient lookup. HyperAST is simultaneously a scientific contribution and the toolkit on which the temporal and polyglot analyses of §4.2.1 and §4.2.2 depend [10, 8, 27]. `https://github.com/HyperAST`

Am I Unique. The public Web platform `https://amiunique.org` to study Web browser diversity and device fingerprinting `https://play.google.com/store/apps/details?id=com.amiunique.amiuniqueapp`, which has collected millions of fingerprints and serves both as a research instrument and as a public education tool [48].

EXADPrinter. A system that enumerates the Android interface surface reachable without permissions and constructs device fingerprints from it [1]. `https://github.com/ExadPrinter`

Fork vulnerability propagation. An implementation of global history analysis that scales to the complete Software Heritage commit graph, together with a public lookup service allowing maintainers and users to determine whether a repository inherits a known unpatched vulnerability [3].

History integrity tooling. A prototype that queries archived superseded states to report whether a repository has a record of destructive history alteration, for use in vetting [21].

Corpora and benchmarks. Two deliverables are datasets rather than tools, and we regard them as first-class outputs: a conformance benchmark for inventory generation derived from our catalogue of divergence cases [4], and a retrospective cohort for validating risk indicators (§4.2.3).

8 Project positioning

8.1 Inria

PIRAT (Rennes) studies the complete attack chain, from intrusion to detection and response, with a strong systems and network security orientation. CANCUN is complementary and upstream: we study how defects and malicious code enter software through its supply chain and how they are distributed at ecosystem scale, whereas PIRAT studies their exploitation and detection on running systems. A doctoral student is co-supervised between the two teams.

PRIVATICS works on privacy in applications and networks, with particular strength in the legal and regulatory dimension of data protection and in network-level tracking. CANCUN's privacy work is deliberately positioned at the level of source code and platform interfaces: we ask which code introduces identifiability and when, rather than which deployed services violate which obligations. The two perspectives are complementary and we anticipate joint work.

SPLiTS (Sophia Antipolis) applies formal methods, including verified compilation, to security properties of web and language runtimes. CANCUN's methods are empirical and operate at ecosystem scale, where formal guarantees are not currently attainable; SPLiTS provides guarantees for components where they are. The natural interface is the formalisation work of §4.1.1.

EVREF works on software maintenance and evolution, with an emphasis on tooling for developers of long-lived systems. CANCUN shares its interest in software evolution but studies it as a security foundation—what history reveals about provenance and integrity—rather than as an object of maintenance.

RAISE develops a research programme at the intersection of software engineering and artificial intelligence, with a focus on scientific reproducibility and variability. CANCUN relies on RAISE's expertise for large-scale empirical methodology, and shares an interest in configuration spaces, which appear in our work as a source of both build variability (§4.1.1) and fingerprintable diversity (§4.2.2).

8.2 IRISA

Within IRISA, CANCUN is one of several teams emerging from the DiverSE research group, and the division of concerns is clear. RAISE addresses artificial intelligence for software engineering and reproducibility; the KHORA team led by A. Blouin addresses software language engineering and model-driven development. CANCUN addresses security and privacy properties of software at ecosystem scale. All three study software systems as their object, with different questions. Collaboration is operationalised through co-supervision of doctoral students, shared seminars, and joint funding, including PEEPS.

8.3 National

Software Heritage (Inria and UNESCO) is our principal scientific infrastructure partner, and the collaboration is active rather than nominal: it is formalised by the Software Heritage Sec contract and has produced joint publications on fork vulnerability propagation, history alteration, and inventory tool accuracy.

ANSSI provides the operational perspective on sovereign auditing requirements and is a partner in the transfer of our supply chain auditing capability.

Télécom Paris (S. Zacchiroli) collaborates on archive-scale mining of development histories, notably on repository integrity.

Groups working on supply chain security in France include teams at LaBRI and IRIT working on software evolution and code analysis. Our distinguishing focus is the combination of ecosystem scale, provenance, and security and privacy properties.

8.4 International

KTH Royal Institute of Technology (M. Monperrus) is the reference group for automated program repair. Our positioning is complementary by design: we deliberately exclude generative patch synthesis from our core programme (§4.2.1) and focus on propagating and transplanting patches that already exist.

North Carolina State University (L. Williams) leads empirical work on supply chain weakness and malicious package detection, and has served on the jury of doctoral work conducted in this laboratory.

SAP Security Research hosted the industrial doctoral work on supply chain attacks that produced the attack taxonomy this project builds on [7, 26], and remains a partner on questions of industrial applicability.

Luxembourg Institute of Science and Technology (J. Cabot) collaborates through PEEPS on interface specification and its divergence from implementation.

Mozilla Foundation and W3C are long-standing collaborators of the team on browser privacy, and the natural route by which fingerprinting results reach platform developers.

9 Short biographies

Walter Rudametkin (team leader) is a Full Professor at the University of Rennes, a member of the Institut Universitaire de France, and a researcher at IRISA and Inria. His research lies at the intersection of software engineering, web security, and online privacy, and he is recognised for pioneering work on browser and device fingerprinting. He co-created the Am I Unique project https://amiunique.org, a reference platform for studying browser diversity that has collected millions of fingerprints, and his recent work extends device fingerprinting to the Android platform. He has published in premier security and Web engineering venues including IEEE S&P, USENIX Security, PETS, and WWW, and received the CNIL-Inria Privacy Protection Award in 2018. Within CANCUN he leads the privacy and fingerprinting axis, and in particular the source-level analysis of fingerprinting surface that connects that work to the project's provenance programme. https://rudametw.github.io/

Olivier Barais is a Full Professor at the University of Rennes and a member of IRISA and the Inria centre at the University of Rennes. His research spans software engineering, model-driven engineering, and software security. He has more than 150 peer-reviewed publications and, over the past decade, has moved decisively towards open-source supply chain security, co-authoring the reference taxonomy of attacks on open-source supply chains and its companion studies of dependency execution and malicious package detection [7, 33, 31]. He is a member of Software Heritage and a scientific lead of the Software Heritage Sec project, and he supervised the doctoral work at the origin of much of the project's supply chain programme. He collaborates actively with industrial partners and is strongly involved in transferring results to the open-source community. https://olivier.barais.fr

Stéphanie Challita is an Associate Professor at the University of Rennes and a researcher at IRISA and Inria. Her research applies model-driven engineering to complex systems, with a specialisation in the reverse engineering of programming interfaces: her doctoral work inferred precise models from cloud interfaces, and she has since worked on the specification, generation, and co-evolution of interface descriptions. She is the principal investigator of the ANR JCJC project PEEPS (2026–2030) on the coherence between requirements, code, and interface documentation. Within CANCUN she leads the formal treatment of declared contracts against observed behaviour (§4.1.1) and contributes the interface-model extraction techniques on which the enumeration of large platform surfaces depends (§4.2.2). https://stephaniechallita.github.io/

Gerson Sunyé is an Associate Professor [HDR] at the University of Rennes and a researcher at IRISA and Inria. His research concerns model-driven engineering, software testing, and the scalability of the infrastructures on which large-scale software analysis depends. His early work formalised the application of design patterns as behaviour-preserving transformations over UML models, and he has since worked extensively on the persistence and querying of very large models, notably through scalable model repositories built over graph and NoSQL backends, and on the testing and benchmarking of distributed systems. Within CANCUN he contributes the persistence and query infrastructure on which ecosystem-scale analysis of the global commit graph rests (§4.1.1, §4.2.1), and the benchmarking methodology behind the two corpora the project delivers as first-class outputs: the conformance suite for inventory generation (§4.1.1) and the retrospective cohort for validating risk indicators (§4.2.3). https://sunye-g.univ-nantes.io/

Olivier Zendra is a Research Scientist [HDR] at Inria. His background is in compilation, runtime systems, and program optimisation, giving him a detailed understanding of the mapping between source code and execution environments. Within CANCUN he brings a transversal concern for efficiency—ensuring that analyses intended to run at ecosystem scale actually can—and his compilation expertise is central to binary-level analysis and to the cross-language interoperability problems that arise throughout the supply chain axis. He coordinates the team's involvement in HiPEAC and co-coordinates the TAP project. https://members.loria.fr/OZendra/

9.1 Strategic local ecosystem and complementary expertise

The scope of CANCUN requires expertise beyond a single team, and we rely on close local collaboration in three areas. For formal methods, and specifically verified compilation and language semantics, we rely on EPICURE. For the theoretical limits and adversarial robustness of the machine learning components we use as engineering tools, we collaborate with Artishau—a collaboration made concrete by the demonstrated fragility of malicious package detectors under adaptive obfuscation [18]. For empirical methodology at ecosystem scale, we work with RAISE and KHORA. These links are operationalised through co-supervision of doctoral students, shared seminars, and joint funding applications.

10 Publications and awards

Awards of the team

Distinctions.

- 2022 — Junior member of the Institut Universitaire de France (IUF) (W. Rudametkin).
- 2018 — L'Oréal–UNESCO *For Women in Science* Young Talents France award (S. Challita).

Paper and competition awards.

- 2026 — Best Student Paper, runner-up, at PETS 2026, for the EXADPrinter paper on permissionless Android fingerprinting [1] (S. Bouhenniche, P. Laperdrix W. Rudametkin).
- TODO: Distinguished paper and artefact awards for work on history analysis at scale (Q. Le Dilavrec's HyperAST/HyperDiff line [10, 8] with O. Barais).
- 2024 — Best Student Paper Award at PETS 2024 for *Client-side and Server-side Tracking on Meta: Effectiveness and Accuracy* [24] (A. El Fraihi, N. Amieur, O. Goga, W. Rudametkin).
- 2024 — Best Paper Award at MADWeb 2024 (NDSS workshop) for *Free Proxies Unmasked: A Vulnerability and Longitudinal Analysis of Free Proxy Services* [28] (N. Mehanna, W. Rudametkin, P. Laperdrix, A. Vastel).
- 2023 — First place, CSAW Europe Applied Research Competition, for *SoK: Taxonomy of Attacks on Open-Source Software Supply Chains* [7], presented by P. Ladisa.
- 2022 — First place, CSAW MENA Applied Research Competition, for DrawnApart (GPU fingerprinting) [9], presented by N. Mehanna and T. Laor.
- 2021 — Mozilla security bug bounty for the DrawnApart GPU fingerprinting vector [9], published at NDSS 2022.
- 2020 — Best Paper Award at MADWeb 2020 for *FP-Crawlers: Studying the Resilience of Browser Fingerprinting to Block Crawlers* [45] (A. Vastel, W. Rudametkin, R. Rouvoy, X. Blanc).
- 2018 — CNIL–Inria Privacy Protection Award for *Beauty and the Beast* [48], IEEE S&P 2016 (P. Laperdrix, W. Rudametkin, B. Baudry).

Most relevant publications of the team in the last five years

Selected from more than 200 publications.

[1] Sihem Bouhenniche, Pierre Laperdrix, and Walter Rudametkin. "EXADPrinter: Semi-Exhaustive Permissionless Device Fingerprinting Within the Android Ecosystem". In: *Proceedings of the 26th Privacy Enhancing Technologies Symposium*. Calgary, Canada, July 2026. URL: https://hal.science/hal-05585898.

[2] Haitam El Hayani, Jolan Philippe, Stéphanie Challita, Olivier Barais, and Benoît Combemale. "Quality assurance in infrastructure as code: Issues, approaches, and open challenges". In: *Journal of Systems and Software* (2026), p. 113080. ISSN: 0164-1212. DOI: https://doi.org/10.1016/j.jss.2026.113080. URL: https://www.sciencedirect.com/science/article/pii/S0164121226003134.

[3] Romain Lefeuvre, Charly Reux, Stefano Zacchiroli, Olivier Barais, and Benoit Combemale. "Did You Forkget It? Detecting One-Day Vulnerabilities in Open-Source Forks with Global History Analysis". In: *Proceedings of the 2026 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses (SCORED '26)*. To appear (October 6, 2026). Final author list, pages, and DOI to be added. Prague, Czech Republic: ACM, Oct. 2026. URL: https://hal.science/hal-05349203.

[4] Alan Prado, Olivier Zendra, Philippe Boinot, and Olivier Barais. "Mind the Gap: How SBOM Specification Ambiguities Lead to Divergent Software Bills of Materials. An Empirical Tool Study". In: *Proceedings of the 2026 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses (SCORED '26)*. To appear (October 6, 2026). Final author list, pages, and DOI to be added. Prague, Czech Republic: ACM, Oct. 2026.

[5] Georges Aaron Randrianaina, Djamel Eddine Khelladi, Olivier Zendra, and Mathieu Acher. "PyroBuildS: Speeding up the exploration of large configuration spaces with incremental build". In: *Journal of Systems and Software* 231 (2026), p. 112592. DOI: 10.1016/j.jss.2025.112592. URL: https://hal.science/hal-05213625.

[6] Maxime Huyghe, Walter Rudametkin, and Clément Quinton. "FP-Rainbow: Fingerprint-Based Browser Configuration Identification". In: *Proceedings of the ACM on Web Conference 2025*. WWW '25. Sydney NSW, Australia: Association for Computing Machinery, 2025, pp. 4325–4335. ISBN: 9798400712746. DOI: 10.1145/3696410.3714699. URL: https://doi.org/10.1145/3696410.3714699.

[7] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. "SoK: Taxonomy of Attacks on Open-Source Software Supply Chains". In: *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2023, pp. 1509–1526. DOI: 10.1109/SP46215.2023.10179304.

[8] Quentin Le Dilavrec, Djamel Eddine Khelladi, Arnaud Blouin, and Jean-Marc Jézéquel. "HyperDiff: Computing Source Code Diffs at Scale". In: *ESEC/FSE 2023 - 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, Dec. 2023, pp. 1–12. DOI: 10.1145/3611643.3616312.

[9] Tomer Laor, Naif Mehanna, Antonin Durey, Vitaly Dyadyuk, Pierre Laperdrix, Clémentine Maurice, Yossi Oren, Romain Rouvoy, Walter Rudametkin, and Yuval Yarom. "DRAWNAPART: A Device Identification Technique based on Remote GPU Fingerprinting". In: *Network and Distributed System Security Symposium*. San Diego, United States, Feb. 2022. DOI: 10.14722/ndss.2022.24093. URL: https://inria.hal.science/hal-03526240.

[10] Quentin Le Dilavrec, Djamel Eddine Khelladi, Arnaud Blouin, and Jean-Marc Jézéquel. "HyperAST: Enabling Efficient Analysis of Software Histories at Scale". In: *ASE 2022 - 37th IEEE/ACM International Conference on Automated Software Engineering*. Oakland, United States: IEEE, Oct. 2022, pp. 1–12. URL: https://inria.hal.science/hal-03764541.

11 References

[11] Linux Foundation. *SPDX: Software Package Data Exchange Specification*. https://spdx.dev. ISO/IEC 5962:2021.

[12] Open Source Security Foundation (OpenSSF). *OpenSSF Scorecard: Security Health Metrics for Open Source*. https://securityscorecards.dev.

[13] OWASP Foundation. *CycloneDX Bill of Materials Specification*. https://cyclonedx.org. ECMA-424.

[14] [TODO : DiverSE Team authors]. "NPM, an Ecosystem Full of Monsters? The Dark Side of Lifecycle Scripts". Under submission. Preprint: [deposit on HAL/arXiv and insert identifier]. 2026.

[15] Paschal C. Amusuo, Kyle A. Robinson, Tanmay Singla, Huiyun Peng, Aravind Machiry, Santiago Torres-Arias, Laurent Simon, and James C. Davis. "ZTDJava: Mitigating Software Supply Chain Vulnerabilities via Zero-Trust Dependencies". In: *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2025, pp. 1294–1306. DOI: 10.1109/ICSE55347.2025.00148.

[16] Russ Cox. "Fifty Years of Open Source Software Supply-Chain Security". In: *Commun. ACM* 68.10 (Sept. 2025), pp. 88–95. ISSN: 0001-0782. DOI: 10.1145/3762635. URL: https://doi.org/10.1145/3762635.

[17] Cybersecurity and Infrastructure Security Agency (CISA). *2025 Minimum Elements for a Software Bill of Materials (SBOM)*. Tech. rep. Public comment draft. CISA, Aug. 2025.

[18] Montaruli, [first name] and others — TO COMPLETE. [*Exact title TO COMPLETE — functionality-preserving obfuscation evading malicious package detectors*]. [TO COMPLETE: venue, authors, pages, DOI]. 2025.

[19] OWASP GenAI Security Project. *Agentic AI — Threats and Mitigations*. Tech. rep. If the team prefers to cite the "OWASP Top 10 for Agentic Applications", substitute its released title and URL here. OWASP Foundation, 2025. URL: https://genai.owasp.org/resource/agentic-ai-threats-and-mitigations/.

[20] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions". In: *Commun. ACM* 68.2 (Jan. 2025), pp. 96–105. ISSN: 0001-0782. DOI: 10.1145/3610721. URL: https://doi.org/10.1145/3610721.

[21] Solal Rapaport, Laurent Pautet, Samuel Tardieu, and Stefano Zacchiroli. "Altered Histories in Version Control System Repositories: Evidence from the Trenches". In: *40th IEEE/ACM International Conference on Automated Software Engineering (ASE 2025)*. IEEE/ACM. Seoul, South Korea, Nov. 2025. URL: https://hal.science/hal-05249034.

[22] Joseph Spracklen, Raveen Wijewickrama, A H M Nazmus Sakib, Anindya Maiti, Bimal Viswanath, and Murtuza Jadliwala. "We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs". In: *34th USENIX Security Symposium (USENIX Security 25)*. USENIX Association, 2025.

[23] David Bors. *CVE-2024-3094 - The XZ Utils Backdoor, a critical SSH vulnerability in Linux*. Pentest Tools blog. 2024. URL: https://pentest-tools.com/blog/xz-utils-backdoor-cve-2024-3094.

[24] Asmaa El Fraihi, Nardjes Amieur, Walter Rudametkin, and Oana Goga. "Client-side and Server-side Tracking on Meta: Effectiveness and Accuracy". In: *Proceedings on Privacy Enhancing Technologies*. Vol. 2024. 3. Bristol, United Kingdom, July 2024, pp. 431–445. DOI: 10.56553/popets-2024-0086. URL: https://hal.science/hal-04665102.

[25] European Parliament and Council of the European Union. *Regulation (EU) 2024/2847 on Horizontal Cybersecurity Requirements for Products with Digital Elements (Cyber Resilience Act)*. Official Journal of the European Union. 2024. URL: https://eur-lex.europa.eu/eli/reg/2024/2847/oj.

[26] Piergiorgio Ladisa. "Securing the Open-Source Software Supply Chain". Industrial (SAP Security Research) doctoral thesis. [VERIFY exact title and HAL identifier]. thesis. Université de Rennes, 2024.

[27] Quentin Le Dilavrec. "Precise temporal analysis of source code histories at scale". thesis. Université de Rennes, Feb. 2024. URL: https://theses.hal.science/tel-04583698.

[28] Naif Mehanna, Walter Rudametkin, Pierre Laperdrix, and Antoine Vastel. "Free Proxies Unmasked: A Vulnerability and Longitudinal Analysis of Free Proxy Services". In: *Workshop on Measurements, Attacks, and Defenses for the Web (MADWeb'24)*. San Diego (CA), United States, Feb. 2024, pp. 1–12. DOI: 10.14722/madweb.2024.23035. URL: https://hal.science/hal-04489166.

[29] NIST. *CVE-2024-3094: Malicious code in XZ Utils*. National Vulnerability Database. 2024. URL: https://nvd.nist.gov/vuln/detail/CVE-2024-3094.

[30] Synopsys. *2024 Open Source Security and Risk Analysis Report (OSSRA)*. Tech. rep. Accessed: 2025. Synopsys Cybersecurity Research Center, 2024. URL: https://www.synopsys.com/software-integrity/resources/analyst-reports/open-source-security-risk-analysis.html.

[31] Piergiorgio Ladisa, Serena Elisa Ponta, Nicola Ronzoni, Matias Martinez, and Olivier Barais. "On the Feasibility of Cross-Language Detection of Malicious Packages in npm and PyPI". In: *Proceedings of the 39th Annual Computer Security Applications Conference (ACSAC '23)*. ACM, 2023. DOI: 10.1145/3627106.3627138.

[32] Piergiorgio Ladisa, Serena Elisa Ponta, Antonino Sabetta, Matias Martinez, and Olivier Barais. "Journey to the Center of Software Supply Chain Attacks". In: *IEEE Security & Privacy* 21.6 (2023), pp. 34–49. DOI: 10.1109/MSEC.2023.3302066.

[33] Piergiorgio Ladisa, Merve Sahin, Serena Elisa Ponta, Marco Rosa, Matias Martinez, and Olivier Barais. "The Hitchhiker's Guide to Malicious Third-Party Dependencies". In: *Proceedings of the 2023 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses* (*SCORED '23*). ACM, 2023, pp. 65–74. DOI: 10.1145/3605770.3625212.

[34] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. "Do Users Write More Insecure Code with AI Assistants?" In: *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. CCS '23. Copenhagen, Denmark: Association for Computing Machinery, 2023, pp. 2785–2799. ISBN: 9798400700507. DOI: 10.1145/3576915.3623157. URL: https://doi.org/10.1145/3576915.3623157.

[35] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, Olivier Barais, and Serena Elisa Ponta. "Towards the Detection of Malicious Java Packages". In: *Proceedings of the 2022 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses* (*SCORED '22*). ACM, 2022. DOI: 10.1145/3560835.3564548.

[36] Chris Lamb and Stefano Zacchiroli. "Reproducible Builds: Increasing the Integrity of Software Supply Chains". In: *IEEE Software* 39.2 (2022), pp. 62–70. DOI: 10.1109/MS.2021.3073045.

[37] Frank Nagle, James Dana, Jennifer Hoffman, Steven Randazzo, and David A. Wheeler. *Census II of Free and Open Source Software — Application Libraries*. Tech. rep. The Linux Foundation & Laboratory for Innovation Science at Harvard, 2022. URL: https://www.linuxfoundation.org/research/census-ii-of-free-and-open-source-software-application-libraries.

[38] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. "Practical Program Repair in the Era of Large Pre-trained Language Models". In: *Proceedings of the 44th International Conference on Software Engineering* (*ICSE*). ACM, 2022.

[39] Nusrat Zahan, Thomas Zimmermann, Patrice Godefroid, Brendan Murphy, Chandra Maddila, and Laurie Williams. "What Are Weak Links in the npm Supply Chain?" In: *2022 IEEE/ACM 44th International Conference on Software Engineering: Software Engineering in Practice* (*ICSE-SEIP*). ACM, 2022, pp. 331–340. DOI: 10.1145/3510457.3513044.

[40] Knut Blind, Mirko Böhm, Pa Grzegorzewska, et al. *The impact of Open Source Software and Hardware on technological independence, competitiveness and innovation in the EU economy*. Final Study Report. European Commission, 2021. DOI: 10.2759/430161.

[41] Gabriel Ferreira, Limin Jia, Joshua Sunshine, and Christian Kästner. "Containing Malicious Package Updates in npm with a Lightweight Permission System". In: *2021 IEEE/ACM 43rd International Conference on Software Engineering* (*ICSE*). IEEE, 2021, pp. 1334–1346. DOI: 10.1109/ICSE43902.2021.00121.

[42] Umar Iqbal, Steven Englehardt, and Zubair Shafiq. "Fingerprinting the Fingerprinters: Learning to Detect Browser Fingerprinting Behaviors". In: *2021 IEEE Symposium on Security and Privacy* (*SP*). IEEE, 2021, pp. 1143–1161. DOI: 10.1109/SP40001.2021.00017.

[43] Open Source Security Foundation (OpenSSF). *Open Source Vulnerability* (*OSV*) *Schema and Database*. https://osv.dev, https://ossf.github.io/osv-schema/. 2021.

[44] United States National Telecommunications and Information Administration (NTIA). *The Minimum Elements For a Software Bill of Materials* (*SBOM*). Tech. rep. United States Department of Commerce, 2021. URL: https://www.ntia.gov/report/2021/minimum-elements-software-bill-materials-sbom.

[45] Antoine Vastel, Walter Rudametkin, Romain Rouvoy, and Xavier Blanc. "FP-Crawlers: Studying the Resilience of Browser Fingerprinting to Block Crawlers". In: *MADWeb'20 - NDSS Workshop on Measurements, Attacks, and Defenses for the Web*. Ed. by Oleksii Starov, Alexandros Kapravelos, and Nick Nikiforakis. San Diego, United States, Feb. 2020. DOI: 10.14722/ndss.2020.23xxx. URL: https://inria.hal.science/hal-02441653.

[46] Antoine Pietri, Diomidis Spinellis, and Stefano Zacchiroli. "The Software Heritage Graph Dataset: Public Software Development Under One Roof". In: *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*. IEEE, 2019, pp. 138–142. DOI: 10.1109/MSR.2019.00030.

[47] Martin Monperrus. "Automatic Software Repair: A Bibliography". In: *ACM Computing Surveys (CSUR)* 51.1 (2018), pp. 1–24.

[48] Pierre Laperdrix, Walter Rudametkin, and Benoit Baudry. "Beauty and the Beast: Diverting Modern Web Browsers to Build Unique Browser Fingerprints". In: *2016 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2016, pp. 878–894. DOI: 10.1109/SP.2016.57.

[49] Peter Eckersley. "How Unique Is Your Web Browser?" In: *Privacy Enhancing Technologies (PETS 2010)*. Vol. 6205. Lecture Notes in Computer Science. Springer, 2010, pp. 1–18. DOI: 10.1007/978-3-642-14527-8_1.